
Concurrencia y Paralelismo

Bloque I: Concurrencia

Mayo 2025

1. Bus [2 puntos]

Un autobús circula por una línea con un cierto número de paradas. La línea es circular, es decir, después de la última parada se vuelve a la primera. Puede suponerse que solo circula un autobús en la línea.

El autobús, al llegar a una parada, avisa a quien esté esperando para subir y bajar, y espera a que los pasajeros suban y bajen del bus. Una vez ha terminado la parada cambia a la siguiente estación.

El código de los threads pasajeros se proporciona como referencia. Implemente la función bus.

```
struct bus_line {
    int current_stop; // Current stop
    int total_stops; // Total number of stops
    pthread_mutex_t m;
    int get_off[STOPS]; // Number of passengers that want leave at that stop
    int load[STOPS]; // Number of passengers that want to enter at that stop
    pthread_cond_t stop[STOPS];
    pthread_cond_t loading;
}

void bus(struct bus_line *bus) {
    ...
}

void passenger(struct bus_line *bus, int from, int to) {
    pthread_mutex_lock(&bus->m);
    if(bus->current_stop != from) {
        bus->load[from]++;
        pthread_cond_wait(&bus->load[from], &bus->m);
        bus->load[from]--;
        if(bus->load[from] == 0) pthread_cond_signal(&bus->loading);
    }
    bus->get_off[to]++;
    pthread_cond_wait(&bus->stop[to], &bus->m);
    bus->get_off[to]--;
    if(bus->get_off[to] == 0) pthread_cond_signal(&bus->loading);
    pthread_mutex_unlock(&bus->m);
}
```

Solución:

```
struct bus_line {
    int current_stop; // Current stop
    int total_stops; // Total number of stops
    pthread_mutex_t m;
    int get_off[STOPS]; // Number of passengers that want leave at that stop
    int load[STOPS]; // Number of passengers that want to enter at that stop
    pthread_cond_t stop[STOPS];
    pthread_cond_t loading;
}

void bus(struct bus_line *bus) {
    pthread_mutex_lock(&bus->m);
    while(true) {
        pthread_mutex_broadcast(bus->get_off[bus->current_stop]);
        pthread_mutex_broadcast(bus->load[bus->current_stop]);
        while(bus->get_off[bus->current_stop] > 0 || bus->load[bus->current_stop] > 0) {
            pthread_cond_wait(&bus->loading, &bus->m);
        }
        bus->current_stop = (bus->current_stop + 1) % bus->total_stops;
    }
    pthread_mutex_unlock(&bus->m);
}

void passenger(struct bus_line *bus, int from, int to) {
    pthread_mutex_lock(&bus->m);
    if(bus->current_stop != from) {
        bus->load[from]++;
        pthread_cond_wait(&bus->load[from], &bus->m);
        bus->load[from]--;
        if(bus->load[from] == 0) pthread_cond_signal(bus->loading);
    }
    bus->get_off[to]++;
    pthread_cond_wait(&bus->stop[to], &bus->m);
    bus->get_off[to]--;
    if(bus->get_off[to] == 0) pthread_cond_signal(bus->loading);
    pthread_mutex_unlock(&bus->m);
}
```

2. Mutex con bloqueo ordenado [1.5 puntos]

Implemente, utilizando los mutex de la librería pthread, un tipo de mutex donde en caso de que tengan que esperar los threads lo bloqueen respetando el orden en el que llegan a la operación lock. Cuando el mutex se libere, solo debería despertarse un thread como máximo.

Para la solución puede usar la cola ya implementada en las funciones `q_init`, `q_insert` y `q_remove`

```
typedef {  
    ...  
} ord_mutex_t;  
  
void q_init(queue q);  
void q_insert(queue q, void *);  
void *q_remove(queue q); // returns NULL if the queue is empty  
  
void ord_mutex_init(ord_mutex_t *m) {  
    ...  
}  
  
void ord_mutex_lock(ord_mutex_t *m) {  
    ...  
}  
  
void ord_mutex_unlock(ord_mutex_t *m) {  
    ...  
}
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);  
int pthread_mutex_lock(pthread_mutex_t *m);  
int pthread_mutex_trylock(pthread_mutex_t *m);  
int pthread_mutex_unlock(pthread_mutex_t *m);  
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);  
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);  
int pthread_cond_signal(pthread_cond_t *cond);  
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

```
typedef {
    pthread_mutex_t m;
    bool locked;
    queue q;
} ord_mutex_t;

void ord_mutex_init(ord_mutex_t *m) {
    pthread_mutex_init(&m->m, NULL);
    m->locked = false;
    q_init(m->q);
}

void ord_mutex_lock(ord_mutex_t *m) {
    pthread_cond_t c;

    pthread_mutex_lock(&m->m);
    if(m->locked) {
        pthread_cond_init(&c, NULL);
        q_insert(m->q, &c);
        pthread_cond_wait(&c, &m->m);
    }
    m->locked=true;
    pthread_mutex_lock(&m->m);
}

void ord_mutex_unlock(ord_mutex_t *m) {
    pthread_mutex_lock(&m->m);
    if((c = q_remove(m->q)) != NULL) {
        pthread_cond_signal(c);
    } else {
        locked = false;
    }
    pthread_mutex_unlock(&m->m);
}
```

3. Línea de procesos [1.5 puntos]

El siguiente módulo permite crear una secuencia de procesos donde cada uno conoce el PID del siguiente.

```
-module(line).
-export([start/1, size/1]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

size(First) ->
    ...

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
    ...
    end,
    loop(Next).
```

Implemente la función `size/1`, que dado el PID del primer proceso devuelva el número de procesos de la línea. Las funciones `start/1` e `init/1` no pueden modificarse.
Solución:

```
-module(line).
-export([start/1, init/1, size/1]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

size(First) ->
    First ! {get_size, self(), 0},
    receive
        {get_size_reply, N} ->
            N
    end.

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
        {get_size, From, N} ->
            case Next of
                none ->
                    From ! {get_size_reply, N + 1};
                _ ->
                    Next ! {get_size, From, N + 1}
            end
    end,
    loop(Next).
```

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria ordinaria

19 de mayo de 2025

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregarán por separado.

[2.5p] 1. Conceptos de paralelismo.

Tras la reintroducción del servicio militar obligatorio, alumnos de la FIC se hacen un fichero de N alegaciones de reclutas de otros años que evitaron su alistamiento. A fin de deducir las características más efectivas de las alegaciones diseñan el siguiente algoritmo:

```
Alegacion A[N]; /* vector de alegaciones */
Caracteristica C[N][M]; /* C[i][j]=caracteristica j en la alegacion i */

leer(A);
for(i = 0; i < N; i++)
    analizar(A[i], C[i]);
resumir(C);
```

donde **analizar** cada alegación para cubrir su vector de características requiere un tiempo costoso, muy variable e impredecible. Se dispone de 8 PCs, de los cuales sólo uno tiene el fichero de datos y conexión a los demás, siendo las comunicaciones muy rápidas.

Responde RAZONADAMENTE a las siguientes cuestiones:

- [0.5p]** Determina qué tipo de descomposición(es) y asignación(es) de tareas aplicarías.
- [1p]** Si en una ejecución con asignación estática el tiempo de **leer** son 5 s, el de **resumir** 25 s y en la fase de análisis los PCs requieren entre 9 y 70 s, con una media de 40 s, calcula explicando los pasos y parámetros que usas:
 - Aceleración
 - Eficiencia paralela.
 - Coste
 - Sobrecarga
- [0.25p]** Supón que un análisis muy rápido permitiese calcular el tiempo que requerirá procesar cada alegación. ¿Cambiarías la descomposición y/o la asignación? Si es así, explica qué estrategia seguirías.
- [0.5p]** En el caso del apartado anterior, si llamamos T_i al número de segundos de análisis para las alegaciones asignadas al PC P_i y el tiempo requerido para que P_0 se las envíe a P_i es $T_i/10$, mientras que los tiempos de **leer** y **resumir** son de 1 s cada una, y no hay ningún coste adicional, ¿cuál es el tiempo total de ejecución del programa suponiendo que todas las comunicaciones son bloqueantes?
- [0.25p]** Si se distribuyen las alegaciones estáticamente entre los 8 PCs cíclicamente por bloques de 3 alegaciones, indica en qué PC y en qué posición relativa dentro del vector local de alegaciones estaría la alegación 58 explicando paso a paso los cálculos que haces.

[2.5p] 2. Diseño de algoritmos paralelos.

La empresa de logística Fast&Furious distribuye periódicamente P tipos de productos a T tiendas de toda Galicia. Para gestionar el pedido, cada tienda indica la cantidad que quiere de cada producto.

Se presenta el algoritmo que usan actualmente para calcular el precio total de los productos distribuidos a cada tienda **ps**, y también la suma del peso de todos los productos del mismo tipo **wp**. También, para un subconjunto de tiendas, imprime el precio que se les debe cobrar.

```
int pedido[T][P], tienda_id;
float pesos[P], precios[P], wp[P], ps[T];

inicializa_datos(pedido, pesos, precios);

... //NOTA: Reparto de datos

memset( ps, 0, T * sizeof(float) );
memset( wp, 0, P * sizeof(float) );

//NOTA: Paralelizar bucle
for (i=0; i<P; i++) {
    for (k=0; k<T; k++) {
        wp[i] += pesos[i] * pedido[k][i];
        ps[k] += precios[i] * pedido[k][i];
    }
}

while ( read( &tienda_id ) ) {
    printf("Los productos de la
           tienda %d cuestan %f euros\n",
           tienda_id, ps[tienda_id]);
}

... //NOTA: Recoleccion de resultados

imprime_resultados ( wp, ps );
```

- La matriz **pedido** de tamaño $T \times P$ contiene las cantidades de cada producto solicitadas por cada tienda. T y P **no tienen por qué ser un múltiplo** del número de procesos.
- El peso unitario de cada uno de los **P** tipos de producto se almacena en un array **pesos**, y el precio en un array **precios**.
- Las funciones `inicializa_datos(...)` e `imprime_resultados(...)` sólo las debe llamar el proceso 0.
- La función `read(...)` la pueden llamar todos los procesos y devuelve la misma secuencia de valores para todos ellos.
- La función `printf` la puede ejecutar cualquier proceso, y debe ejecutarse antes de la recolección de los resultados.
- El creador del algoritmo puso algunas NOTAS sobre cómo proceder.

Diseña e implementa la paralelización del algoritmo propuesto paralelizado siguiendo las especificaciones anteriores. **Comenta y justifica** las decisiones tomadas. Para simplificar el código:

- Se puede asumir que el entorno MPI ya está inicializado y existen las variables **rango** y **numprocs** con los valores correspondientes en cada proceso.
- Los buffers de envío y recepción pueden solaparse.
- Puedes definir abreviaturas, como por ejemplo MCW para MPI_COMM_WORLD.

```
int MPI_Barrier(MPI_Comm comm)
int MPI_Bcast(void *buffer, int count, MPI_Datatype dt,
              int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
               MPI_Datatype dt, MPI_Op op,
               int root, MPI_Comm comm)
```

SOLUCIÓN

1. a) En este problema hay una **descomposición funcional** con las tareas de lectura, análisis y resumen. La segunda tarea a su vez tiene una **descomposición de dominio sobre el vector de alegaciones y la matriz de características**, esta última por filas.

La asignación de tareas durante el análisis es **dinámica centralizada** (maestro-esclavo) ya que sólo un PC puede comunicarse con todos los demás y hay un desequilibrio de carga potencial muy importante si hacemos una distribución estática de cualquier tipo sin poder saber de antemano el coste de procesamiento de cada alegación.

- b) Si se hace una asignación estática, los 8 PCs pueden analizar alegaciones en paralelo. Por otra parte, si el tiempo medio de procesamiento son 40 s/PC , el tiempo secuencial debería ser de $40 \times 8 = 320\text{ s}$, que sumados a las partes que se ejecutan secuencialmente dan $5 + 320 + 25 = \boxed{350\text{ s}}$.

El tiempo paralelo será la suma de los tiempos secuenciales más el tiempo de análisis más largo de un PC, lo que son $5 + 70 + 25 = \boxed{100\text{ s}}$. A partir de ahí,

- $Ac = 350/100 = \boxed{3,5}$
- $Ef = 3,5/8 = 0,44 = \boxed{44\%}$
- $Coste = 100\text{ s} \times 8 = \boxed{800\text{ s}}$
- $Sobrecarga = Coste - T_{seq} = 800 - 350 = \boxed{450\text{ s}}$

- c) La descomposición del análisis **seguiría siendo de dominio**, pero se haría una **asignación estática** al conocer el coste. Lo más sencillo para equilibrar la carga sería que fuese cíclica tras ordenar las alegaciones por su coste de procesamiento. También podría hacerse un reparto irregular con bloques consecutivos de distinto tamaño, pero con costes similares.

- d) Las partes secuenciales del programa, **leer** y **resumir**, tardan 2 s en ejecutarse.

Por otra parte el PC P_i , $1 \leq i \leq 7$, inicia su análisis de T_i segundos después de que se hayan enviado sus bloques a los PCs 1 a i , con lo que acaba su análisis tras $(\sum_{j=1}^i T_j)/10 + T_i$ segundos. Por su parte, el PC 0 podrá iniciar su análisis tras enviar los trabajos a los otros PCs, con lo que finalizará tras $(\sum_{j=1}^7 T_j)/10 + T_0$ segundos.

El tiempo de ejecución para la fase de análisis será el máximo de estos tiempos, puesto que todos tienen que acabar para que se inicie el resumen. Por tanto la expresión final sería

$$2 + \max \left(\max \left\{ \left(\sum_{j=1}^i T_j \right) / 10 + T_i, 1 \leq i \leq 7 \right\}, \left(\sum_{j=1}^7 T_j \right) / 10 + T_0 \right)$$

- e) La alegación 58 se encuentra dentro del bloque $\lfloor 58/3 \rfloor = 19$, el cual al ser el reparto cíclico, se encuentra en el PC $19 \bmod 8 = 3$.

Dentro de su bloque de 3 elementos, la alegación está en la posición relativa $58 \bmod 3 = 1$.

Por otra parte, antes del bloque 19 hay otros $\lfloor 19/8 \rfloor = 2$ bloques en la memoria local de su PC, cada uno de ellos de 3 alegaciones. Por tanto la posición local será $2 \times 3 + 1 = 7$.

2. A continuación se detallan varias formas de resolver este ejercicio. Recordamos que el enunciado indica en negrita “**Comenta y justifica**” las decisiones tomadas.

a) Reparto por filas de la matriz pedido con padding

- Se define `filas_alloc` para minimizar el uso de memoria y evitar que todos los procesos reserven la matriz completa.
- Se define `filas_ef` para evitar que el último proceso haga cálculos con basura.
- Se hace un reparto por filas de la matriz `pedido` (Scatter), por lo que cada proceso necesita los arrays `pesos` y `precios` completos (Bcast).
- Cada proceso calculará una parte del array `ps` y resultados parciales de `wp`. Por tanto serán necesarios al final un Gather y un Reduce, respectivamente.
- En el bloque del `printf`, como debe ejecutarse antes de la recolección, los procesos tienen que determinar si `tienda_id` pertenece a su bloque local, y cuál es su posición local.

```
float pesos[P], precios[P];
float wp[P];

int filas = ceil(T/numprocs);
int filas_alloc = rango ? filas : filas*numprocs;
int filas_ef = rango < (numprocs-1)
                ? filas : T-filas*(numprocs-1);

float pedido[ filas_alloc ][P];
float ps [ filas_alloc ];

if (!rango)
    inicializa_datos(pedido, pesos, precios);

MPI_Scatter(pedido, filas*P, MPI_INT,
            pedido, filas*P, MPI_INT, 0, MCW);
MPI_Bcast(pesos, P, MPI_FLOAT, 0, MCW);
MPI_Bcast(precios, P, MPI_FLOAT, 0, MCW);

memset( ps, 0, filas * sizeof(float) );
memset( wp, 0, P * sizeof(float) );

for (i=0; i<P; i++) {
    for (k=0; k<filas_ef; k++) {
        wp[i] += pesos[i] * pedido[k][i];
        ps[k] += precios[i] * pedido[k][i];
    }
}

int tienda_id;
while ( read( &tienda_id ) ) {
    if (rango == tienda_id / filas)
        printf("Los productos de la tienda %d cuestan %f euros\n",
               tienda_id, ps[tienda_id % filas]);
    MPI_Barrier(MPI_COMM_WORLD);
}

MPI_Gather(ps, filas, MPI_FLOAT,
           ps, filas, MPI_FLOAT, 0, MCW);
MPI_Reduce(wp, wp, P, MPI_FLOAT, MPI_SUM, 0, MCW);

if (!rango)
    imprime_resultados ( wp, ps );
```

1. Reserva de memoria (padding, reparto por filas):

`filas`: tamaño de bloque
`filas_alloc`: tamaño de reserva de memoria.
 T con padding para el root,
 "filas" para todos los demás
`filas_ef`: bloque corregido para evitar las
 posiciones añadidas en el padding

2. Reparto correcto de datos:

Si se reparte por filas la matriz "pedido",
cada proceso necesita su parte (Scatter)
y los arrays "pesos" y "precios" completos (Bcast)
"ps" y "wp" no se dividen: son datos de salida

3. Distribución de la carga de trabajo:

Si se reparte por filas la matriz `pedido[k][i]`,
sólo se divide el bucle "k".
"filas_ef" evita que el último proceso haga
operaciones con basura si `T%numprocs > 0`.

4. Como el array "ps" se encuentra repartido,

hay que mirar si "tienda_id" pertenece al proceso
y calcular su posición local en el array.
Opcionalmente, se añade una barrera para sincronizar
las iteraciones del bucle.

5. Recolección de resultados:

Los datos de salida son "wp" y "ps".
"ps" está dividido y hay que hacer un Gather
"wp" contiene sumas parciales y hay que reducirlo

b) Alternativa a padding: el proceso 0 calcula los elementos no repartidos

```
float pesos[P], precios[P];
float wp[P];

int filas = T / numprocs;
int filas_alloc = rango ? filas : T;
int resto = T % numprocs;

float pedido[ filas_alloc ][P];
float ps [ filas_alloc ];

... // igual que en padding por filas

for (i=0; i<P; i++) {
    for (k=0; k<filas; k++) {
        wp[i] += pesos[i] * pedido[k][i];
        ps[k] += precios[i] * pedido[k][i];
    }
}

if (!rango && resto) {
    for (i=0; i<P; i++) {
        for (k=T-resto; k<T; k++) {
            wp[i] += pesos[i] * pedido[k][i];
            ps[k] += precios[i] * pedido[k][i];
        }
    }
}

int tienda_id;
while ( read( &tienda_id ) ) {
    int proc = tienda_id / filas;
    if (rango == proc)
        printf("Los productos de la tienda %d cuestan %f euros\n",
            tienda_id, ps[tienda_id % filas]);
    else if (!rango && proc >= numprocs)
        printf("Los productos de la tienda %d cuestan %f euros\n",
            tienda_id, ps[tienda_id]);
    MPI_Barrier(MPI_COMM_WORLD);
}

... // igual que en padding por filas
```

1. Reserva de memoria (reparto por filas):

filas: tamaño de bloque
filas_alloc: tamaño de reserva de memoria.
T para el root,
"filas" para todos los demás

2. El bucle de computación calcula en paralelo las "filas x numprocs" primeras filas.

3. El proceso 0, que tiene los datos de pedido completos, realiza los cálculos para las filas que no han sido repartidas. Esto no afecta al Gather ni al Reduce posteriores.

La condición "&& resto" es opcional, ya que si resto es 0 el bucle interno no iterará.

4. Esta parte también cambia ligeramente. Las tiendas que queden fuera del rango de procesos pertenecen al proceso 0 y tiene su información almacenada en las últimas posiciones de "ps".

c) Reparto por columnas de la matriz pedido

- El reparto por columnas de la matriz es más complejo, porque la información no está contigua en memoria (Scatter de cada fila).
- Cada proceso necesita un subconjunto de pesos y precios (Scatter).
- Cada proceso calculará resultados parciales de ps y una parte del array wp. Por tanto serán necesarios al final un Reduce y un Gather, respectivamente.
- En el bloque del printf, como debe ejecutarse antes de la recolección, los procesos deben sumar sus valores parciales para la tienda solicitada y que uno de ellos se encargue de la impresión.

```
float ps[T];

int cols = ceil(P/numprocs);
int cols_alloc = rango ? cols : cols*numprocs;
int cols_ef = rango<(numprocs-1)
               ? cols : P-cols*(numprocs-1);

float pedido[T][ cols_alloc ];
float wp [ cols_alloc ];
float pesos[ cols_alloc ], precios[ cols_alloc ];

if (!rango)
    inicializa_datos(pedido, pesos, precios);

for (int i=0; i<T; i++)
    MPI_Scatter(pedido[i], cols, MPI_INT,
               pedido[i], cols, MPI_INT, 0, MCW);
MPI_Scatter(pesos, bloque, MPI_FLOAT,
            pesos, bloque, MPI_FLOAT, 0, MCW);
MPI_Scatter(precios, bloque, MPI_FLOAT,
            precios, bloque, MPI_FLOAT, 0, MCW);

memset( ps, 0, T * sizeof(float) );
memset( wp, 0, cols * sizeof(float) );

for (i=0; i<cols_ef; i++) {
    for (k=0; k<T; k++) {
        wp[i] += pesos[i] * pedido[k][i];
        ps[k] += precios[i] * pedido[k][i];
    }
}

int tienda_id;
float ptienda;
while ( read( &tienda_id ) ) {
    MPI_Reduce(&ps[tienda_id], &ptienda, 1, MPI_FLOAT,
              MPI_SUM, 0, MCW);
    if (rank == 0)
        printf("Los productos de la tienda %d cuestan %f euros\n",
               tienda_id, ptienda);
}

MPI_Gather(wp, cols, MPI_FLOAT,
           wp, cols, MPI_FLOAT, 0, MCW);
MPI_Reduce(ps, ps, T, MPI_FLOAT, MPI_SUM, 0, MCW);

if (!rango)
    imprime_resultados ( wp, ps );
```

1. Reserva de memoria (padding, reparto por columnas):

cols: tamaño de bloque
cols_alloc: tamaño de reserva de memoria.
P con padding para el root,
"cols" para todos los demás
cols_ef: bloque corregido para evitar las
posiciones añadidas en el padding

2. Reparto correcto de datos:

Si se reparte por columnas la matriz "pedido",
hay que repartir fila a fila (Scatter)
y cada proceso solo necesita una parte de
los arrays "pesos" y "precios".

"ps" y "wp" no se dividen: son datos de salida

3. Distribución de la carga de trabajo:

Si se reparte por columnas la matriz pedido,
sólo se divide el bucle "i".

"cols_ef" evita que el último proceso haga
operaciones con basura si P%numprocs > 0.

4. Como el array "ps" contiene sumas parciales

hay que reducir esa suma para poder imprimir el
valor correcto.

5. Recolección de resultados:

Los datos de salida son "wp" y "ps".
"wp" está dividido y hay que hacer un Gather
"ps" contiene sumas parciales y hay que reducirlo